BITACORA DE LABORATORIO

Que la transferencia no deje dinero en el aire

Saga orquestada y circuit breaker en Mexi Banco. Punto de partida: 03-gateway-discovery-balanceo; ejecucion local con Docker Compose.

1. El problema antes de arreglarlo

En la version S07, transferencia cargaba al origen y luego abonaba al destino mediante dos peticiones HTTP independientes. Cada servicio confirmaba su propia transaccion; una falla del segundo POST no revertia el primero.

```
Cargo confirmado: saldo origen=800.00; se apaga cuenta durante la pausa
Respuesta del POST:
{"timestamp":"2026-10-15T17:45:49.805+00:00","path":"/api/transferencias","status":504,"
error":"Gateway Timeout","requestId":"bf9ccb01-27"}
HTTP 504
Saldos despues:
{
  "clabe": "002180009208630901",
  "saldo": 800.00
}
{
  "clabe": "002180009208630902",
  "saldo": 500.00
}
Filas de transferencia (debe ser 0 para estas CLABEs):
0
```

Figura 1. Captura de pantalla: cargo, error 504 y saldos antes del cambio.

El total paso de 1500 a 1300: faltaron 200. El gateway dio 504 a los 15 segundos, pero la pausa seguia activa dentro de transferencia. Al regresar cuenta, origen seguia en 800 y destino en 500; ni una fila explicaba la falla.

2. La saga orquestada

El servicio transferencia es el orquestador. Guarda cada estado en PostgreSQL con saveAndFlush antes de cruzar la red. El metodo principal no lleva una transaccion local larga, porque esta no podria deshacer el cargo remoto.

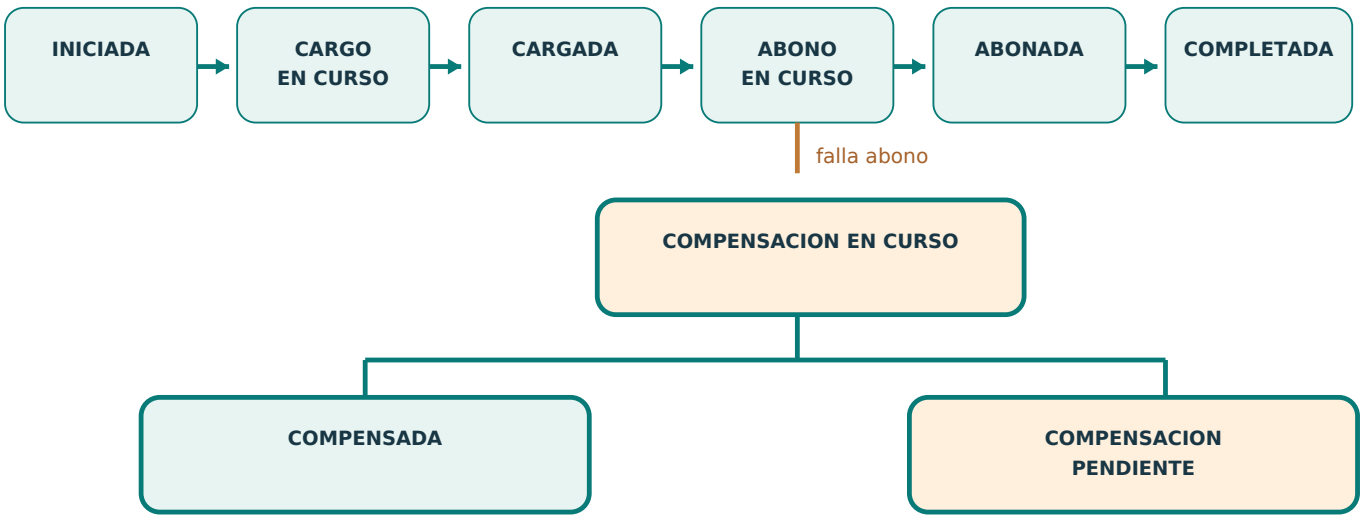


Figura 2. Camino normal y rama de compensacion. Los estados EN CURSO delimitan operaciones cuyo resultado podria ser incierto tras un reinicio.

Que se compensa y que no

Validacion: si falla, RECHAZADA sin mover dinero. Cargo: si no hay confirmacion, CARGO_INCIERTO; no se repite a ciegas. Abono: si falla tras el cargo, se abona el monto al origen. Asiento despues del abono: no se devuelve dinero ya recibido; queda REGISTRO_PENDIENTE. Los avisos son de mejor esfuerzo.

3. Codigo clave

```
avanzar(t, EstadoTransferencia.CARGADA,
    "Cargo confirmado en cuenta origen");
avanzar(t, EstadoTransferencia.ABONO_EN_CURSO, null);
destino = cuentas.abonar(claveDestino, monto);
return compensar(t, "Fallo el abono al destino: " + mensaje(error));
```

El controlador responde 201 cuando completa, 409 si se compenso y 202 si queda trabajo pendiente. El JSON contiene id, estado y detalle para que el cliente no reciba un 500 generico.

4. La compensacion funcionando

Configure FALLAR_ABONO_CLABE con la CLABE destino. La inyeccion ocurre antes del POST de abono y deja cuenta encendida, de modo que la devolucion al origen si puede ejecutarse.

```
8:  "saldo": 500
10:POST /api/transferencias con falla simulada del abono:
11:"{\"id\":2,\"claveOrigen\":\"002180009208662111\",\"claveDestino\":\"0021800092086621
12\",\"monto\":200,\"fecha\":\"2026-10-15T17:50:36.939803118Z\",\"estado\":\"COMPENSADA\
",\"detalle\":\"Fallo el abono al destino: Falla simulada del abono para la CLABE 002180
009208662112; monto devuelto al origen\"}\nHTTP 409\n"
12:Saldos finales:
15:  "saldo": 1000.00
19:  "saldo": 500.00
21:Fila durable:
22:2|COMPENSADA|Fallo el abono al destino: Falla simulada del abono para la CLABE 002180
009208662112; monto devuelto al origen
26:    "tipo": "TRANSFERENCIA_RECIBIDA",
27:    "monto": 200.00,
28:    "referencia": "COMPENSACION transferencia #2"
31:    "tipo": "TRANSFERENCIA_ENVIADA",
32:    "monto": -200.00,
36:    "tipo": "DEPOSITO",
37:    "monto": 1000.00,
```

Figura 3. Captura de pantalla: compensacion, saldos y movimientos.

El saldo regreso a 1000 y el destino permanecio en 500. El historial registra el cargo y la devolucion con referencia COMPENSACION transferencia #2. Se reutilizo TRANSFERENCIA_RECIBIDA porque el enum original de movimiento no tiene un tipo DEVOLUCION.

Fragmento de la compensacion

```
avanzar(t, EstadoTransferencia.COMPENSACION_EN_CURSO, motivo);
origenRestaurado = devolverConReintentoSeguro(t);
MovimientoRemoto devolucion = movimientos.registrar(
    t.getClaveOrigen(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
    t.getMonto(), origenRestaurado.saldo(),
    "COMPENSACION transferencia #" + t.getId());
avanzar(t, EstadoTransferencia.COMPENSADA,
    motivo + "; monto devuelto al origen");
```

El estado se consulta tambien en GET /api/transferencias/{id} y el historial en GET /api/movimientos?clabe=....

5. Cuando tampoco se puede devolver

Repeti la pausa de 20 segundos, observe el cargo y apague cuenta antes del abono. Fallaron tanto el abono como la devolucion. La fila no quedo atascada en CARGADA: paso a COMPENSACION_PENDIENTE.

```
4:  "saldo": 1000
8:  "saldo": 500
11:Cargo confirmado: origen=800.00; se detiene cuenta
14:HTTP 504
15:Estado con cuenta apagada: COMPENSACION_PENDIENTE
16:9|COMPENSACION_PENDIENTE|Fallo el abono al destino: No hay instancias disponibles de
cuenta; no se envio la peticion HTTP; devolucion NO confirmada: Cuenta no disponible: ci
rcuito abierto; intenta de nuevo mas tarde
17:Saldos tras recuperar cuenta (la devolucion NO se reintenta a ciegas):
20:  "saldo": 800.00
24:  "saldo": 500.00
```

Figura 4. Captura de pantalla: error 504, compensacion pendiente y saldos.

Para terminarla en un sistema real, un proceso de conciliacion compararia el estado de la saga con el saldo y los movimientos de cuenta. Solo entonces devolveria lo faltante. La API de cuenta no acepta Idempotency-Key para abonos: si un POST da timeout, quiza se aplico. Repetirlo sin comprobarlo puede devolver dos veces.

Punto opcional: timeout y espera creciente segura

Las llamadas HTTP tienen timeout de conexion de 2 s y de lectura de 5 s. La compensacion admite tres intentos con esperas de 250 y 500 ms solo si Eureka no encontro instancias o el circuito rechazo localmente la llamada: en ambos casos no salio un POST. Timeouts y 5xx de un POST enviado quedan pendientes, sin reintento ciego.

```
catch (CircuitoAbiertoException | CuentaSinInstanciasException e) {
    if (intento == 3) throw e;
    Thread.sleep(esperaMs);  // 250 ms, luego 500 ms
    esperaMs *= 2;
}
```

6. Circuit breaker manual

CuentaCircuitBreaker envuelve consultas, cargos y abonos. Cuenta fallas de disponibilidad (red, 5xx, sin instancias), pero no 404/422 de negocio. Tras tres fallas abre; ocho segundos despues admite una sola prueba en HALF_OPEN. Si responde, vuelve a CLOSED.

```
11:Intento 1
12:HTTP 503 tiempo=2.392255s
14:Intento 2
15:HTTP 503 tiempo=0.987681s
17:Intento 3
18:HTTP 503 tiempo=2.037164s
20:Intento 4
21:HTTP 503 tiempo=0.028884s
23:Prueba tras recuperar cuenta y vencer la espera del circuito:
24:HTTP 201 tiempo=0.306327s
27:CIRCUITO cuenta CLOSED: falla de disponibilidad 1/3
28:CIRCUITO cuenta CLOSED: falla de disponibilidad 2/3
29:CIRCUITO cuenta CLOSED: falla de disponibilidad 3/3
30:CIRCUITO cuenta CLOSED -> OPEN
31:CIRCUITO cuenta OPEN: llamada rechazada sin HTTP
32:CIRCUITO cuenta OPEN -> HALF_OPEN
33:CIRCUITO cuenta HALF_OPEN -> CLOSED
```

Figura 5. Captura de pantalla: intentos y transiciones del circuit breaker.

La cuarta respuesta fue casi inmediata porque el circuito ya no golpeo a cuenta. Al recuperarla, el intento de prueba cerro el circuito y la transferencia termino con HTTP 201.

7. Lo que la saga no da: aislamiento

Entre el cargo y la compensacion, otra consulta puede ver el saldo de origen en 800, aunque despues vuelva a 1000. La saga recupera consistencia eventual, pero no oculta estados intermedios. Tampoco ofrece exactamente-una-vez sin soporte idempotente del servicio cuenta.

Verificacion y fuentes

Pruebas Java: ./mvnw test con salida 0; integracion: Docker, curl, logs y PostgreSQL. Fuentes: consigna local del Lab S08; codigo base Mexi Banco, rama 03-gateway-discovery-balanceo, commit 87cfbe9. Implementacion y observaciones de esta bitacora: propias.